

Train–Validation–Test Split

What is Data Splitting?

Data splitting is the process of dividing the original dataset into separate parts in order to:

- train a machine learning model,
- tune its parameters,
- objectively evaluate its performance on unseen data.

In machine learning, **a model must never be trained and evaluated on the same data**, as this leads to misleading results and overfitting.

Main Dataset Splits

A dataset is usually divided into **three parts**:

Training Set

- Used to **train the model**
- The model learns patterns and relationships from this data
- Typically **60–80%** of the dataset

 Example:

A linear regression model learns its weights using the training set.

Validation Set

- Used for **model tuning and hyperparameter selection**
- Helps choose:
 - learning rate
 - number of epochs
 - model complexity
- The model **does not learn** from this data



Example:

We test different learning rates and pick the one with the lowest validation error.



Test Set

- Used **only once** for final evaluation
- Represents **completely unseen data**
- Provides an unbiased estimate of model performance



Important:

Never tune the model using the test set.



Common Split Ratios

Training	Validation	Test
70%	15%	15%
80%	10%	10%
60%	20%	20%

The exact ratio depends on:

- dataset size
 - task complexity
-



Why Not Just Train and Test?

If we skip the validation set, we often:

- tweak parameters,
- compare models,
- adjust configurations



This unintentionally adapts the model to the test data.

This problem is called **data leakage**, and it invalidates evaluation results.

Overfitting

Overfitting occurs when:

- training error is very low,
- validation/test error is high.

The model memorizes the training data instead of learning general patterns.

 Typical sign:

- Training error ↓
- Validation error ↑

The validation set helps detect this early.

Practical Data Splitting (Python Example)

Using `scikit-learn`:

```
from sklearn.model_selection import train_test_split # 80% training, 20%
temporary X_train, X_temp, y_train, y_temp = train_test_split( X, y,
test_size=0.2, random_state=42 ) # Split remaining 20% into validation and test
sets X_val, X_test, y_val, y_test = train_test_split( X_temp, y_temp,
test_size=0.5, random_state=42 )
```

Why Use `random_state` ?

- Ensures **reproducibility**
 - Same code → same data split
 - Critical for:
 - debugging
 - experiments
 - collaboration
-

Summary

- **Training set** → learn patterns
- **Validation set** → tune parameters
- **Test set** → final, unbiased evaluation

 Proper data splitting:

- prevents overfitting
- avoids data leakage
- is a fundamental ML best practice